

Migrating from Plain Authentication to JWT Authentication

Complete Step-by-Step Checklist (React + Node.js + PostgreSQL + Vercel)

The following are **only the implementation steps** required to convert a normal username/password authentication into JWT-based authentication.

PART 1 - Backend (Node.js)

Step 1

Install the JWT package.

- Install `jsonwebtoken`
 - Keep `bcrypt` for password verification
-

Step 2

Create a JWT Secret.

- Generate a long random secret.
- Add it as an Environment Variable in Vercel.
- (Optional) Add it to the local `.env` file for local testing.

Environment Variable

```
JWT_SECRET=<your secret>
```

Step 3

Import JWT

Modify `login.js`

Add the JWT import.

Step 4

Authenticate the user as before

No changes.

Continue to

- Read Email
 - Read Password
 - Query PostgreSQL
 - Compare password using bcrypt
-

Step 5

Generate a JWT

After successful password verification

- Create a payload
- Select only required fields

Example payload

- id
- name
- email
- role (optional)

Do NOT include

- password

- password hash
 - sensitive information
-

Step 6

Configure Token Expiry

While creating the token

Specify

- 1 hour
- 30 minutes
- 1 day

depending on application requirement.

Step 7

Return the Token

Modify the Login Response.

Instead of returning only

```
success
message
user
```

Return

```
success
message
token
user
```

Step 8

Create Authentication Middleware

Create a new file

```
api/  
  authenticateToken.js
```

Responsibilities

- Read Authorization Header
- Extract Bearer Token
- Verify JWT
- Reject invalid tokens
- Reject expired tokens
- Attach decoded user to request
- Continue to API

Step 9

Protect API Endpoints

Modify every protected API.

Examples

```
employees.js  
products.js  
orders.js  
profile.js
```

Instead of allowing direct access

Require token verification before processing.

Step 10

Handle Authentication Errors

Return proper HTTP Status Codes

Examples

```
401 Unauthorized
```

Missing Token

```
403 Forbidden
```

Invalid Token

```
Token Expired
```

Expired Token

PART 2 - Frontend (React)

Step 11

Modify Login Component

File

```
Login.jsx
```

Changes

- Receive JWT Token
- Receive User Object
- Store Token
- Store User
- Show Login Success

- Redirect User (optional)
-

Step 12

Store JWT

Store JWT in browser storage.

Current implementation

```
localStorage
```

Key

```
token
```

Step 13

Store User Information

Store only

- id
- name
- email
- role (optional)

Do NOT store

- password
 - salary
 - personal data
-

Step 14

Logout

During logout

Remove

```
token  
user
```

from Local Storage.

Step 15

Send JWT with Every Protected Request

Whenever calling protected APIs

Include

```
Authorization  
  
Bearer <token>
```

header.

Step 16

Protect React Pages

Create

```
ProtectedRoute
```

(or equivalent logic)

Responsibilities

- Check if token exists
 - Redirect to Login if missing
 - Allow page access if logged in
-

Step 17

Use User Information

Read

```
localStorage
```

to

- Display Username
- Show Welcome Message
- Display User Profile
- Control Menu Visibility

Step 18 (Optional)

Hide Admin Features

Use stored user information to

- Hide Admin Menu
- Hide Delete Buttons
- Hide Settings

Important

Frontend checks are only for improving the user experience.

They are **not** security.

PART 3 - Request Flow

Step 19

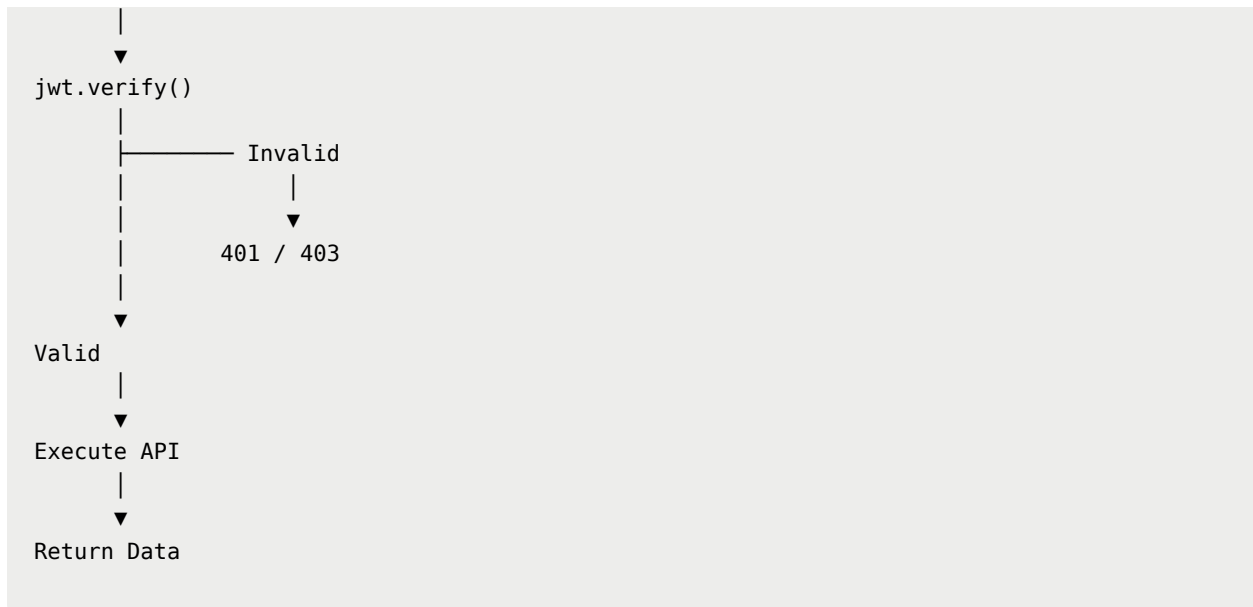
User Login Flow


```
graph TD
    User --> React_Login_Form[React Login Form]
    React_Login_Form --> POST_login[POST /login]
    POST_login --> Node_Login_API[Node Login API]
    Node_Login_API --> Find_User[Find User]
    Find_User --> bcrypt_compare[bcrypt.compare()]
    bcrypt_compare --> Generate_JWT[Generate JWT]
    Generate_JWT --> Return_Token_User[Return Token + User]
    Return_Token_User --> React_stores_Token_User[React stores Token + User]
```

Step 20

Protected API Flow

```
graph TD
    React --> Read_Token[Read Token]
    Read_Token --> Authorization_Bearer[Authorization: Bearer <token>]
    Authorization_Bearer --> Protected_Node_API[Protected Node API]
    Protected_Node_API --> authenticateToken_js[authenticateToken.js]
```



PART 4 - Files Modified

Backend

Existing Files Modified

- `package.json`
- `login.js`

Existing Files (No Major Changes)

- `register.js`
- `db.js`
- `cors.js`

New File

- `authenticateToken.js`

Environment

- `.env` (Local)

- Vercel Environment Variables (`JWT_SECRET`)
-

Frontend

Existing Files Modified

- `Login.jsx`

Existing Files Used

- `config.js`

Optional New Files

- `ProtectedRoute.jsx`
 - `AuthContext.jsx` (optional for larger applications)
-

Packages Used

Backend

- `bcrypt`
- `jsonwebtoken`
- `pg`

Frontend

No additional packages were required for the JWT implementation itself.

Final Architecture

